Web and Data Engineering

Kapitel 01: Organisation und Einleitung

Prof. Dr. Michael Granitzer

Speaker notes

Diese Vorlesung führt in das Web Engineering als Disziplin ein, die heute untrennbar mit Data Engineering verbunden ist. Während Web Engineering die systematische Konstruktion stabiler, skalierbarer Online-Systeme adressiert, liefert Data Engineering die notwendigen Pipelines, um diese Systeme intelligent und wertvoll zu machen.

Ziel dieser Vorlesung

Moderne Web-Systeme verlangen systematisches Engineering - über Architektur, Teams, Daten und AI hinweg. Diese Einheit klärt, **warum** das so ist und **was** Sie in diesem Kurs dafür lernen werden.

Leitfragen

- Warum ist das Web mehr als ein Informationskanal?
- Was macht Web-Systeme komplex - technisch und organisatorisch?
- Warum reicht Coding allein nicht - und was tritt an seine Stelle?
- Wie verändert AI die Rolle von Web Engineers?
- Warum gehört die Datenperspektive dazu?

Speaker notes

Der Fokus liegt auf dem "Engineering"-Aspekt: Während Programmieren das Erstellen von Funktionalität beschreibt, befasst sich Engineering mit der Beherrschung von Komplexität über den gesamten Lebenszyklus hinweg. In einer Welt, in der KI-Agenten und massive Datenmengen den Takt vorgeben, ist das Verständnis von Architektur und Systematik wichtiger als die Kenntnis einzelner Framework-Details.

Das Web als Infrastruktur

Leitfrage: Warum ist das Web mehr als nur ein Informationskanal, nämlich eine technische und gesellschaftliche Kerninfrastruktur?

Das Web als Infrastruktur

Das World Wide Web ist heute weit mehr als eine Sammlung von Webseiten. Es ist die zentrale Infrastruktur für digitale Kommunikation, Plattformen, datengetriebene Anwendungen - und zunehmend auch für autonome AI-Systeme.

Diese Infrastruktur hat sich in mehreren Paradigmenwechseln entwickelt. Jeder Schritt hat neue Engineering-Probleme erzeugt, die diesen Kurs motivieren.

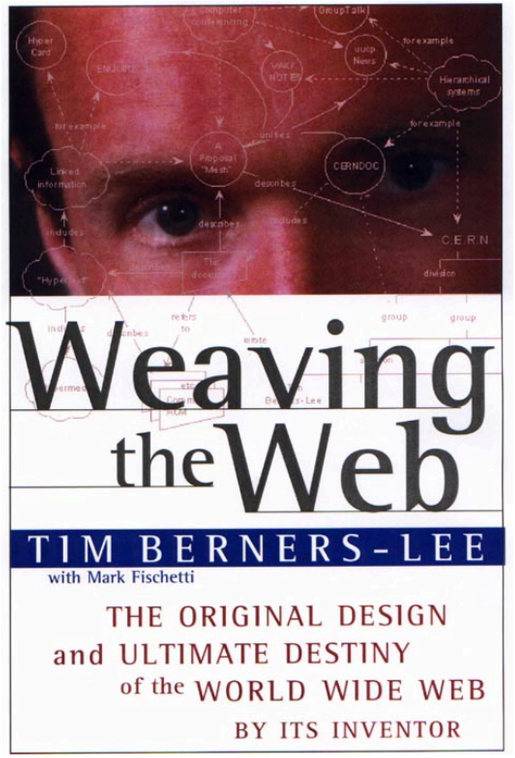
Speaker notes

Die Entwicklung des Webs von 1989 bis heute ist ein Sprung von einer reinen Dokumentenablage für Wissenschaftler zu einer globalen Ausführungsumgebung. Jeder Paradigmenwechsel (Web 1.0, 2.0, 3.0) löste neue technische Herausforderungen aus: Zuerst die Adressierung (URL), dann die Interaktivität (JavaScript/Zustand), schließlich die massive Skalierung (Cloud) und heute die autonome Interaktion durch Agenten.

Von der Vision zum Netz (Web 1.0)

> **Problem:** Wie macht man Wissen maschinengestützt zugänglich?

- 1969: `ARPANET` zeigt, dass verteilte Netze praktisch funktionieren
- 1989: Tim Berners-Lee schlägt am CERN das `World Wide Web` vor
- 1991: das erste öffentliche Webangebot geht online
  - statische Dokumente mit **Hyperlinks**
  - **HTTP** als Protokoll
  - **HTML** als Auszeichnungssprache
  - **URI** als Adressierungsschema



Offene Protokolle und verteilte Infrastruktur werden zum Fundament - aber das Web liefert zunächst nur **statische Dokumente**.

Speaker notes

Vannevar Bushs Vision des "Memex" (1945) beschrieb bereits assoziative Verknüpfungen von Wissen, lange bevor die Technik dafür existierte. Tim Berners-Lee kombinierte diese Idee des Hypertexts mit der dezentralen Netzstruktur des ARPANET. Die entscheidende Neuerung war, dass Dokumente auf verschiedenen Rechnern liegen konnten und über einen universellen Identifikator (URI) verknüpft wurden.

Von Dokumenten zu Anwendungen (Web 1.0 -> Web 2.0)

> **Problem:** Wie wird das Web interaktiv und kommerziell nutzbar?

- 1995: `JavaScript` und serverseitige Technologien (`PHP`, `CGI`) ermöglichen dynamische Seiten
- 1995-2000: `Dotcom-Boom` - E-Commerce (`Amazon`, `eBay`), Webmail, Online-Banking
- 2001: der Dotcom-Crash - es überlebten nur Systeme, die technisch skalierbar gebaut wurden
- 2001: Das Web 2.0 wird propagiert:
  - Benutzer werden zu Produzenten von Inhalten (Prosumer) und neue Geschäftsmodelle entstehen
  - Daten und Netzwerkeffekte werden zum zentralen Wettbewerbsvorteil
  - Perpetual Beta - Software wird kontinuierlich weiterentwickelt und verbessert
  - Rich User Experience - Rich Client Applications

Interaktivität erzeugt **Zustand, Sicherheitsfragen und Serverlogik** - das Web braucht plötzlich Software-Architektur.



Speaker notes

Der Dotcom-Crash (2001) wirkte wie ein Filter: Nur Unternehmen mit technisch soliden, skalierbaren Architekturen überlebten. Hier entstand das Verständnis, dass Web-Anwendungen "Software as a Service" sind, die kontinuierlich betrieben und aktualisiert werden müssen, statt als fertige Produkte (Boxed Software) ausgeliefert zu werden.

Von Anwendungen zu Plattformen (Web 2.0)

> **Problem:** Wie wird das Web zur universellen Betriebsplattform?

- 2006: `AWS` startet - Cloud-Infrastruktur wird on-demand verfügbar
- 2007: `iPhone` macht das Web allgegenwärtig; Responsive Design wird Pflicht
- 2010er: `Microservices` + `Kubernetes` - kontinuierliches Deployment zum Standard; APIs als Integrationsschicht
  - HTML 5, CSS 3, JavaScript ES6+ - Rich Client Applications werden zum Standard
  - AJAX, JSON, REST APIs - Daten werden zum zentralen Element
  - Unterschiede zu Desktop Programmen reduziert sich

Verteilte Systeme, Skalierung und Teamkoordination werden zu **zentralen Engineering-Problemen**. Das Web ist nicht mehr Anwendung, sondern **Betriebsplattform**.

Speaker notes

Mit der Einführung von AWS (2006) verschob sich das Engineering-Problem von der Hardware-Beschaffung hin zur Orchestrierung virtueller Ressourcen. Microservices und Kubernetes ermöglichen es heute, dass Teams unabhängig voneinander Teile eines Gesamtsystems aktualisieren können, ohne den Betrieb zu unterbrechen – eine Grundvoraussetzung für Plattformen mit Millionen Nutzern.

Von Plattformen zu datengetriebenen Systemen (Web 2.0 -> Web 3.0)

> **Problem:** Wie nutzt man die Daten, die Web-Systeme erzeugen?

- Plattformen erzeugen kontinuierlich Daten - Klicks, Käufe, Suchanfragen
- Personalisierung (`Netflix`, `Spotify`) wird vom Zusatzfeature zur **Kernfunktion**
- `Data Pipelines` entstehen als eigenstaendige Infrastruktur; neue Rollen wie `Data Engineer`
- Maschinelles Lernen und diskriminative KI Algorithmen werden integraler Bestandteil von Web-Systemen
  - z.B. Empfehlungsdienste, Suchmaschinen, etc.
  - Rankings und Algorithmic Bias
  - Filter Bubbles

Data Engineering wird integraler Bestandteil von Web-Systemen - genau deshalb verbindet dieser Kurs beide Perspektiven.

Speaker notes

Daten sind heute der Treibstoff der Plattformökonomie. Web-Systeme erzeugen kontinuierliche Log-Streams, die in Echtzeit verarbeitet werden müssen, um Personalisierung und smarte Funktionen zu ermöglichen. Das erfordert eine enge Kopplung von Web-Backends und Data-Pipelines, was neue Anforderungen an Schnittstellendesign und Datenkonsistenz stellt.

Vom menschlichen Nutzer zum AI-Agenten (Web 3.0)

Problem: Wer nutzt das Web in Zukunft - und wie?

- AI-Agenten nutzen zunehmend **Web-basierte Tools**: Suchmaschinen, Webseiten-Inhalte, REST-APIs
- Protokolle wie das `Model Context Protocol (MCP)` standardisieren Nutzung durch Agenten
- Agenten **orchestrieren Ketten von Web-Interaktionen** autonom
- das Web wird zur **Ausführungsumgebung für autonome Systeme**, nicht nur für menschliche Nutzer

Agent-Aktion	Web-Technologie
Informationssuche	Web Search APIs
Seiteninhalt lesen	HTTP + HTML Parsing
Daten abrufen	REST / GraphQL APIs
Aktionen ausführen	Web APIs mit Auth
Tool-Integration	MCP-Server über HTTP

Konsequenz (vermutlich): Web-APIs und -Dienste müssen nicht nur für Menschen, sondern auch für **maschinelle Konsumenten** robust, dokumentiert und sicher sein. Das

Speaker notes

Die Nutzung des Webs durch KI-Agenten erfordert eine "semantische Entkopplung". Agenten interessieren sich nicht für Layout oder Design (CSS), sondern für strukturierte Daten und berechenbare API-Zustände. Protokolle wie das Model Context Protocol (MCP) zielen darauf ab, diese Interaktion zwischen LLMs und Web-Tools zu standardisieren, wodurch das Web zur "Werkbank" für Agenten wird.

Die Paradigmenwechsel im Überblick



Jeder Paradigmenwechsel erzeugt neue Engineering-Probleme — und neue Anforderungen an Web-Systeme.

Jeder Paradigmenwechsel im Web hat neue Engineering-Probleme erzeugt. Mit AI-Agenten steht der nächste bevor: Das Web wird nicht nur Plattform für Menschen, sondern Ausführungsumgebung für autonome Systeme. Die Relevanz des Webs liegt darin, dass hier offene Protokolle, Anwendungslogik und Datenströme zu einem universellen Ausführungsraum zusammenkommen - für menschliche und maschinelle Akteure gleichermaßen.

Speaker notes

Die Pointe ist die wachsende Nutzerschicht: Menschen, Browser, Apps und zunehmend Agenten. Damit werden Protokolle und Schnittstellen noch wichtiger als einzelne UI-Details.

Was Web-Systeme komplex macht

Leitfrage: Was genau macht Web-Systeme so komplex, dass einfaches Programmieren nicht ausreicht?

Definition Web Engineering

> *Web Engineering ist die Anwendung systematischer und quantifizierbarer Ansätze für Anforderungsbeschreibung, Entwurf, Implementierung, Test, Betrieb und Wartung qualitativ hochwertiger Web-Anwendungen.*

Kerngedanke

Web Engineering beschreibt sowohl eine praktische Ingenieurstätigkeit als auch eine wissenschaftliche Disziplin.

Speaker notes

Web Engineering unterscheidet sich vom klassischen Software Engineering durch die spezifischen Anforderungen des Webs: Offenheit, Heterogenität der Endgeräte, unvorhersehbare Lastspitzen und die Notwendigkeit kontinuierlicher Updates. "Systematisch" bedeutet hier, dass Designentscheidungen (z.B. Caching-Strategien) auf klaren Kriterien basieren, statt auf Intuition.

Was sind Web-Anwendungen?

Typische Merkmale

- Nutzung über Web-Protokolle und Browser
- verteilte Komponenten
- Kombination aus UI, Logik und Datenhaltung
- häufig kontinuierliche Weiterentwicklung

Typische Unterschiede

- informationsorientiert vs. transaktionsorientiert
- einfache Sites vs. komplexe Plattformen
- monolithisch vs. verteilt
- rein webbasiert vs. stark datengetrieben

Speaker notes

Diese Kategorisierung hilft dabei, die passenden Engineering-Methoden zu wählen. Ein informationsorientiertes System (Data Lake, Dokumentation) stellt andere Anforderungen an SEO und Caching als ein transaktionsorientiertes System (Bank, Shop), bei dem Konsistenz und Sicherheit im Vordergrund stehen. Verteilte Architekturen werden meist dort notwendig, wo Skalierung oder Team-Autonomie im Monolithen nicht mehr möglich sind.

Skala und Heterogenität des Webs

Das Web heute: Skala und Heterogenität

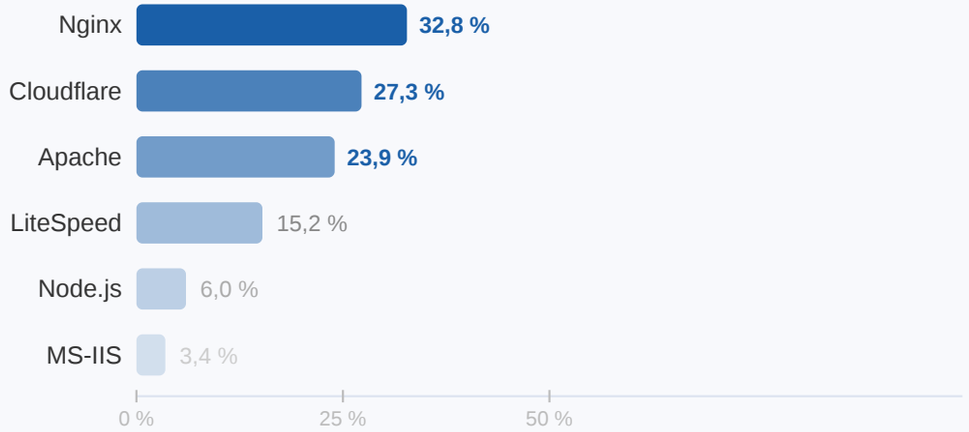
≈ 1,1 Mrd.
Websites weltweit
Internet Live Stats, 2024

16,9 Mio.
analysierte Sites (HTTP Archive)
Web Almanac 2024

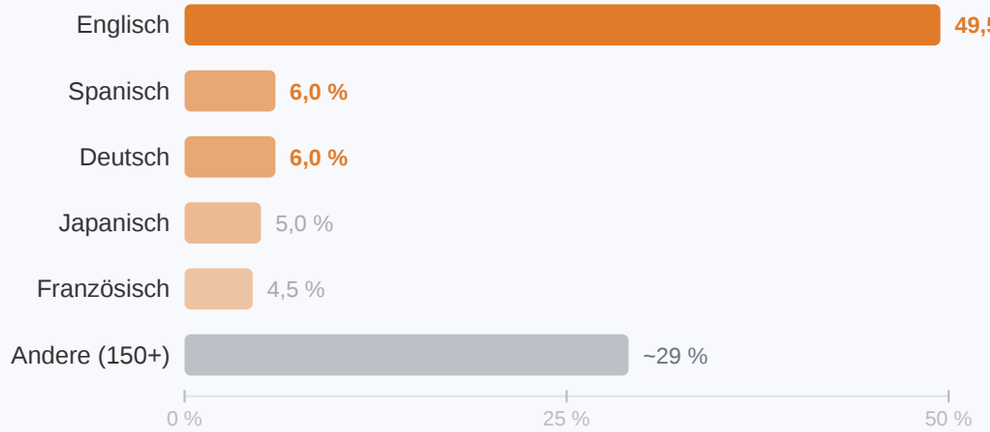
2,6 MB
Median Desktop (71 HTTP-Requests)
Web Almanac 2024

249 Plattformen
erkannte CMS · 51 % aller Sites
Web Almanac 2024

Web-Server Technologien (W3Techs 2024)



Sprachen der Web-Inhalte (W3Techs 2024)



Quellen: HTTP Archive Web Almanac 2024 (Viscomi et al., Nov. 2024) · W3Techs 2024 · Internet Live Stats 2024. Daten beziehen sich auf öffentlich erreichbare Websites.

Source: Eigene Visualisierung auf Basis von HTTP Archive Web Almanac 2024 (Viscomi et al.) und W3Techs 2024. Daten: öffentlich erreichbare Websites, Erhebungszeitraum Juni–November 2024.

Das Web ist riesig und hochgradig heterogen: Es umfasst mehr als eine Milliarde Sites, nutzt hunderte verschiedener Technologien und bedient Inhalte in über 150 Sprachen.

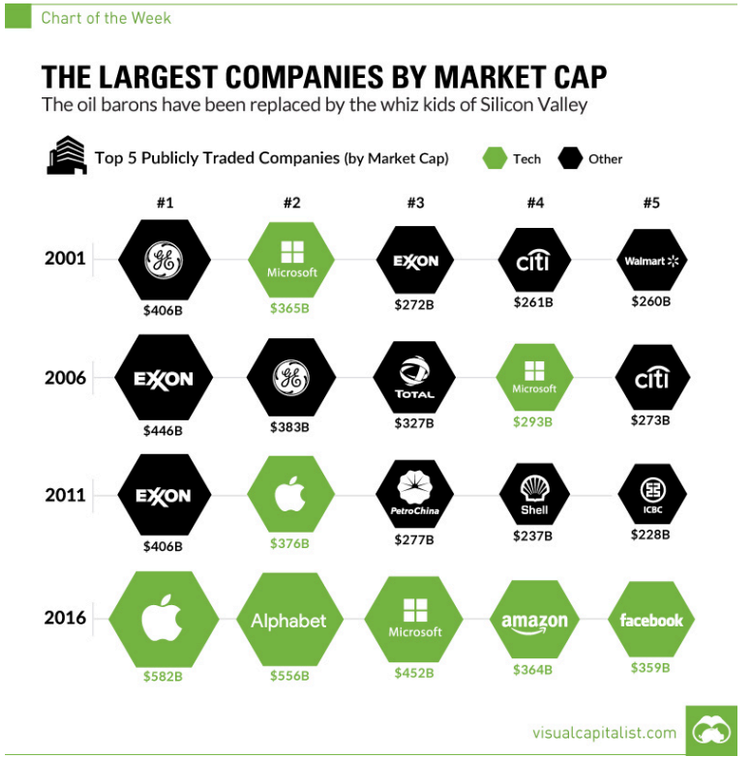
Diese Heterogenität ist eine der zentralen Ursachen für die Komplexität von Web-Systemen.

Speaker notes

Die Zahlen machen die Herausforderung greifbar: 17 Millionen Sites wurden allein im HTTP Archive Crawl 2024 erfasst – mit einer Median-Seitengröße von 2,6 MB und 71 HTTP-Requests. Gleichzeitig zeigt die Verteilung der Web-Server-Technologien (Nginx, Cloudflare, Apache, LiteSpeed, ...) und Inhaltssprachen (über 150 Sprachen), dass kein einheitlicher Standard existiert. Web-Systeme müssen in dieser heterogenen Welt korrekt funktionieren, was Anforderungen an Protokollkonformität, Performance und Internationalisierung stellt. Quelle: HTTP Archive Web Almanac 2024 (Viscomi et al.) und W3Techs 2024.

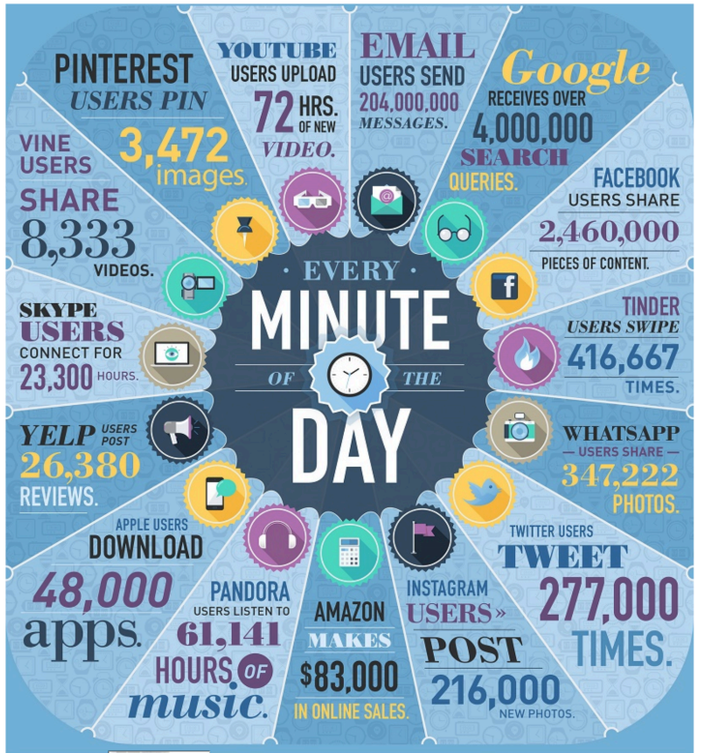
Plattformökonomie

Reichweite und Marktwert digitaler Plattformen entstehen über Web- und Dateninfrastrukturen. Wertschöpfung hängt oft daran, Datenflüsse systematisch zu erfassen und zu nutzen.



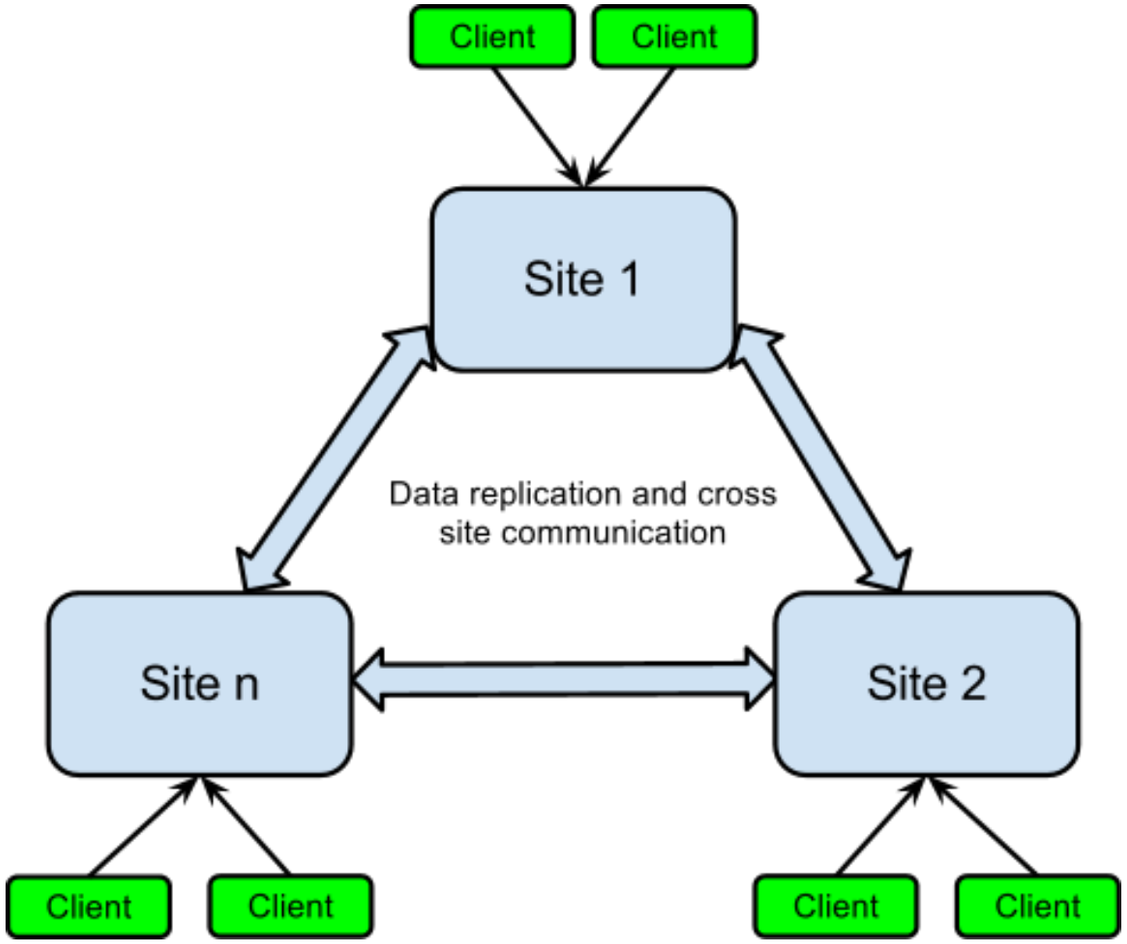
Daten als wirtschaftlicher Hebel

Digitale Plattformen wachsen nicht primär durch bessere Technik, sondern durch systematische Nutzung von Daten. Wer Datenflüsse besser versteht, kann Produkte schneller anpassen, Nutzer gezielter bedienen und neue Geschäftsmodelle erschließen.



Backend-Komplexität am Beispiel Spotify

Moderne Web-Produkte bestehen aus vielen gekoppelten Diensten. Skalierung, Deployment und Betrieb werden zur Architekturfrage.

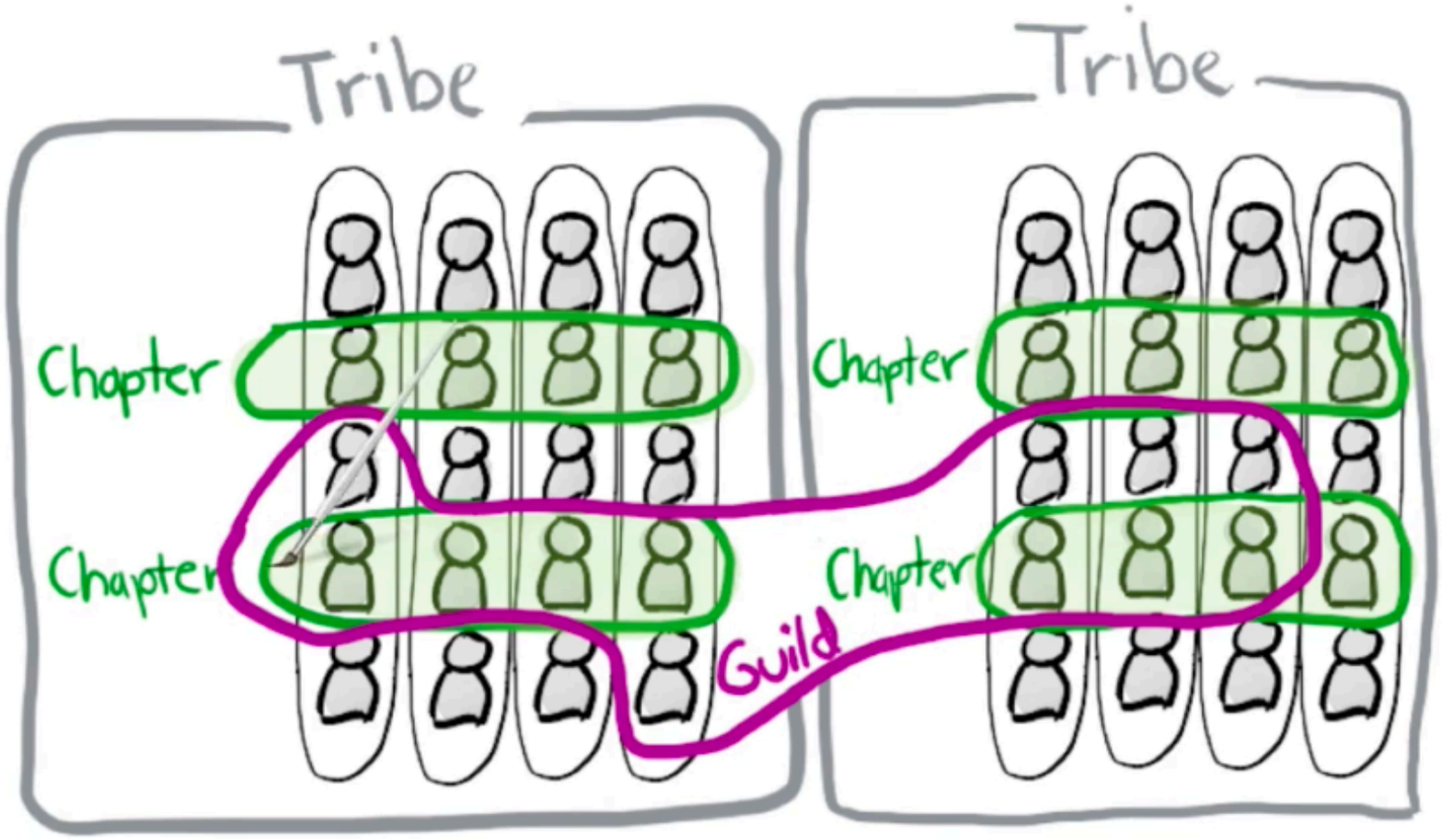


Speaker notes

Ein System wie Spotify besteht aus hunderten von Microservices, die über das Netzwerk kommunizieren. Komplexität entsteht hier nicht im Code eines einzelnen Dienstes, sondern im "Zwischenraum": Service Discovery, Fehlertoleranz (Circuit Breaking), Netzwerklatenz und die Konsistenz von Daten über hunderte Instanzen hinweg.

Teamkomplexität am Beispiel Spotify

Technische Komplexitaet trifft auf verteilte Teamstrukturen. Architektur wird auch zum Mittel der Koordination.

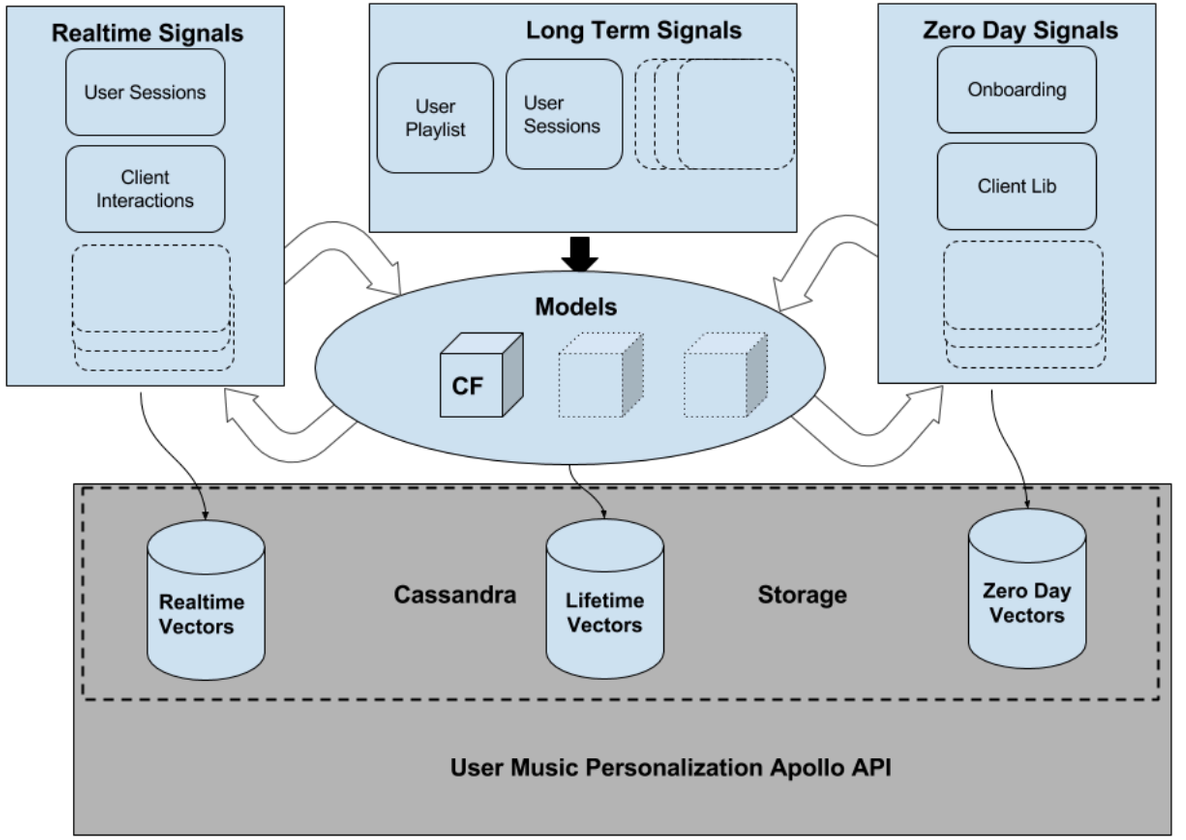


Speaker notes

Conways Gesetz (Conway’s Law) besagt, dass die Architektur eines Systems die Kommunikationsstrukturen der Organisation widerspiegelt. Spotify nutzt "Squads" und "Tribes", um Autonomie zu fördern. Das Web Engineering muss diese Autonomie technisch unterstützen, indem es klare Schnittstellen und stabile Plattform-Infrastrukturen bereitstellt.

AI-Integration am Beispiel Spotify

Personalisierung ist kein Zusatzfeature, sondern Teil der Systemarchitektur. Datenpipelines, Modelle und Auslieferung müssen mit Web-Produktlogik zusammenspielen. Damit verschiebt sich Komplexität von der UI in Infrastruktur, Daten und Betrieb.



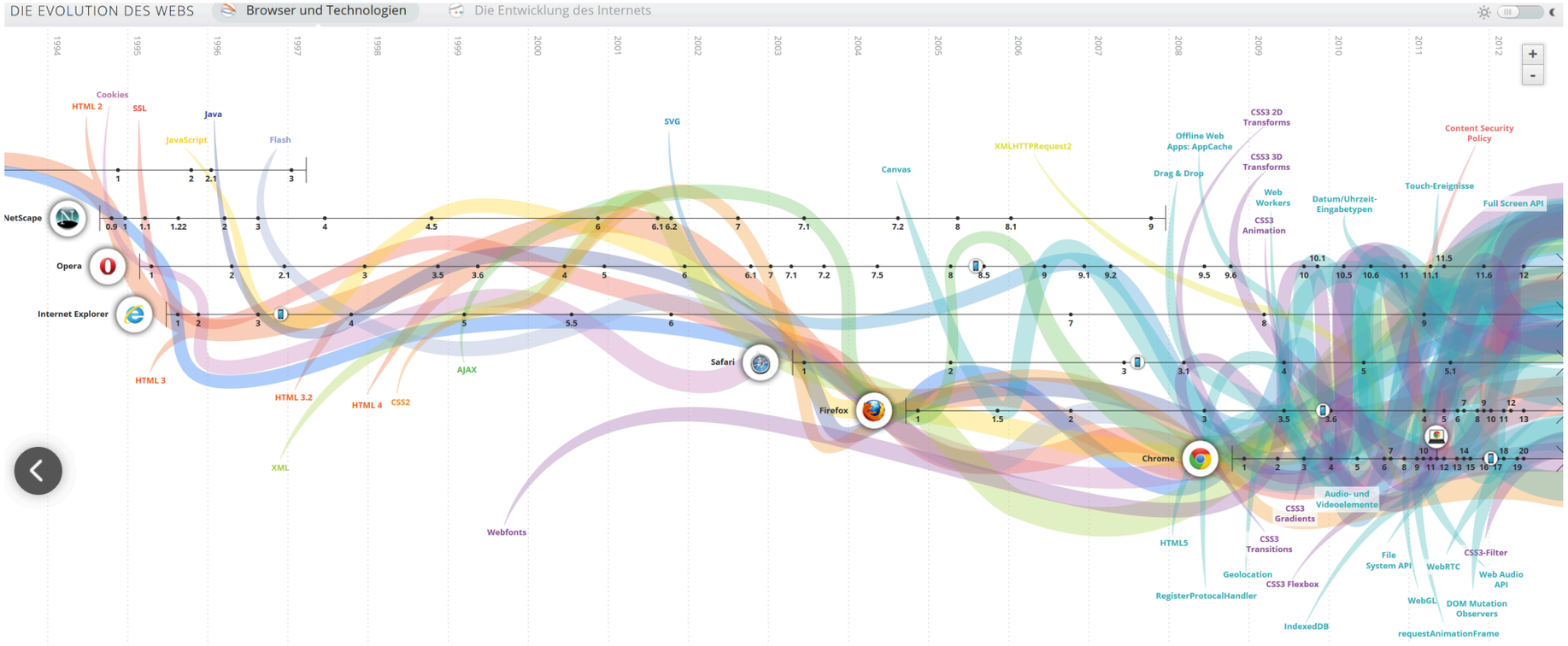
Speaker notes

Personalisierung in Echtzeit ist eine der größten Herausforderungen des modernen Web Engineering. Sie erfordert eine nahtlose Integration von Modell-Inferenz (ML) in den Web-Request-Pfad. Dabei dürfen die harten Qualitätsziele von Web-Systemen (Latenz im Millisekundenbereich) nicht verletzt werden.

Komplexität ist der Normalzustand

Technische Dimension

- Frontend, Backend und Persistenz ändern sich nicht unabhängig
- Sicherheit, Verfügbarkeit und Performance sind Querschnittsthemen
- Standards und Schnittstellen entscheiden über Interoperabilität



Organisatorische Dimension

- Teams arbeiten parallel an gekoppelten Teilsystemen
- Änderungen müssen trotz laufendem Betrieb möglich bleiben
- Qualität entsteht über Prozesse, nicht nur über einzelne Code-Artefakte

Komplexität in Web-Systemen entsteht aus dem Zusammenspiel von Schichten, verteilten Teams und kontinuierlicher Evolution. Diese Komplexität ist kein Ausnahmezustand - sie ist der Normalfall reifer Web-Plattformen.

Vom Coding zum Engineering

Leitfrage: Wenn Web-Systeme so komplex sind - was braucht es über das reine Programmieren hinaus?

Von der Komplexität zur Methode

Was Coding allein nicht löst

- ein einzelnes Feature berührt oft mehrere Schichten gleichzeitig
- Sicherheitslücken, Performanceprobleme und Inkompatibilitäten entstehen zwischen Komponenten
- ohne gemeinsame Methodik entstehen Widersprüche zwischen parallelen Teams

Was Engineering ergänzt

- explizite Anforderungen statt impliziter Annahmen
- bewusste Architekturentscheidungen statt zufällig gewachsener Struktur
- definierte Prozesse für Test, Deployment und Wartung
- Qualitätskriterien, die vor der Implementierung feststehen

Speaker notes

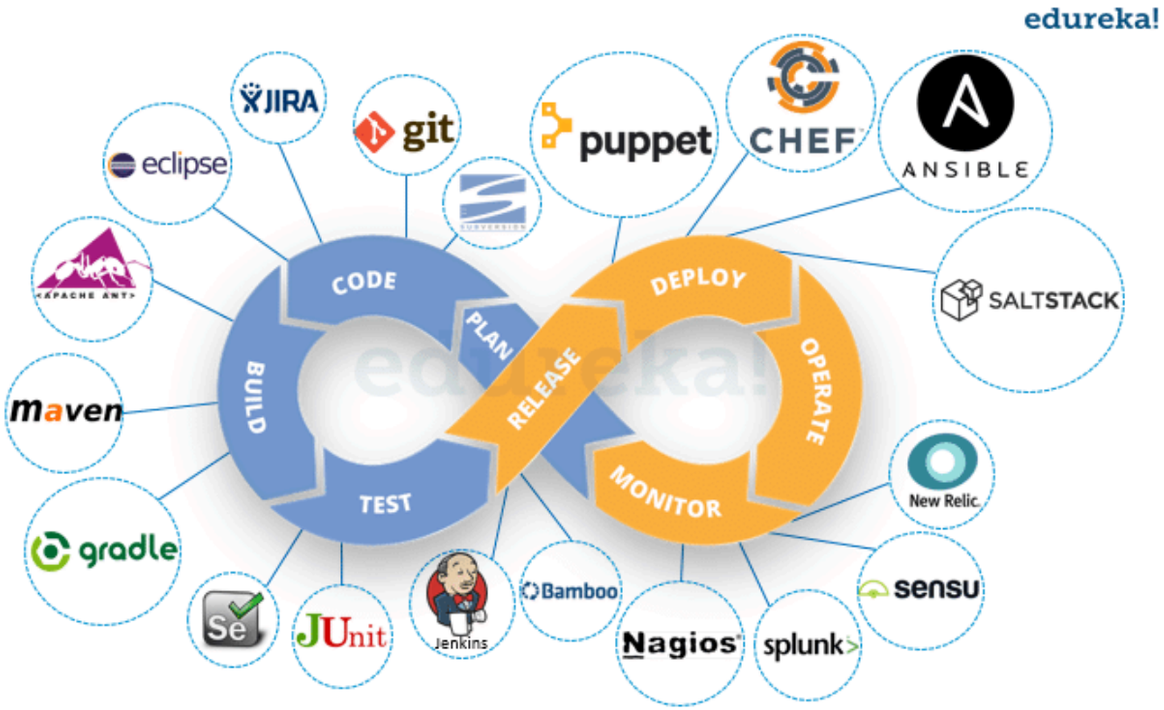
Während reine Programmierung oft lösungsorientiert und ad-hoc stattfindet, fokussiert sich Engineering auf die Herstellbarkeit und Wartbarkeit. Das bedeutet: Entscheidungen werden dokumentiert, Testabdeckung wird erzwungen und der Betrieb (Deployment) ist Teil des Entwurfsprozesses, nicht ein afterthought. Das macht das System unabhängig von einzelnen Entwicklern.



Web Engineering als Entwicklungsprozess

Entwicklungszyklus

- Anforderungen, Entwurf, Implementierung und Betrieb müssen zusammen gedacht werden
- Web Engineering ist prozessorientiert, nicht nur technologieorientiert
- Entwicklung, Deployment und Wartung bilden einen zusammenhängenden Lebenszyklus
- Automatisierung, Automatisierung, Automatisierung



Der Lebenszyklus einer modernen Web-Anwendung ist zirkulär: Durch "Observability" (Monitoring, Logging) fließen Daten aus dem Betrieb direkt zurück in die Anforderungsphase. Automatisierung (CI/CD) ist hier kein Komfortmerkmal, sondern die

Standards als Bedingung für Skalierung

Rolle von Standards



- skalierbare Web-Systeme brauchen gemeinsame technische Grundlagen
- Standards schaffen Interoperabilität zwischen Browsern, Servern und Werkzeugen
- Institutionen wie das W3C treiben diese Entwicklung organisatorisch voran

Konsequenz: Standards wie HTML, CSS und HTTP sichern die "Permissionless Innovation" und Interoperabilität. Sie verhindern ein fragmentiertes Web proprietärer Technologien und ermöglichen es erst, dass verteilte Systeme weltweit zusammenarbeiten können. :::

> Web Engineering wird dort notwendig, wo verteilte Technik, Daten, Betrieb und Teamkoordination nicht mehr sauber durch isoliertes Coding beherrschbar sind. Methoden, Prozesse und Standards machen Komplexität handhabbar.

Speaker notes

Web Engineering wird notwendig, sobald Web-Systeme den Rahmen eines experimentellen Prototyps verlassen. Ab diesem Punkt entscheiden Architektur, strukturierte Prozesse und die Einhaltung von Standards über die wirtschaftliche Nachhaltigkeit und technische Wartbarkeit des Systems.

Web Engineering im Zeitalter von AI

Leitfrage: Wenn AI immer mehr Implementierungsarbeit übernimmt, welche Fähigkeiten werden für Web Engineers wichtiger statt weniger wichtig?

Ausgangsthese

Die Routine des Codings wird zunehmend delegierbar. Dadurch wird die eigentliche Engineering-Arbeit wichtiger: Problemzuschnitt, Architektur, Qualitätskriterien, Risikobewertung und Aufsicht über AI-generierte Artefakte.

Speaker notes

Dies beschreibt einen fundamentalen wirtschaftlichen Shift in der Industrie: Da die Kosten für reine Code-Generierung gegen Null tendieren, verschiebt sich die menschliche Wertschöpfung in Richtung Systemdesign. Entwickler werden zu System-Architekten, die KI-Ressourcen orchestrieren statt selbst jede Zeile zu schreiben.

Evidenz: AI übernimmt bereits spürbare Teile der Implementierung

Produktivität und Delegation

- Microsoft berichtet im Juni 2025 aus drei Feldexperimenten mit 4.867 Entwicklerinnen und Entwicklern im Mittel von +26.08% erledigten Aufgaben mit AI-Unterstützung.
- Anthropic analysierte im April 2025 500.000 coding-bezogene Interaktionen; bei Claude Code wurden 79% als stärker automatisierte Nutzung klassifiziert.

Wo die Automatisierung zuerst greift

- In derselben Anthropic-Analyse dominierten Websprachen wie JavaScript, TypeScript, HTML und CSS.
- User-facing App- und UI/UX-Aufgaben gehörten zu den häufigsten Anwendungsfällen.
- Daraus lässt sich plausibel ableiten, dass gerade einfache Fullstack- und UI-nahe Implementierung früh delegierbar wird.

Von Layer-Spezialisierung zu breiterer Verantwortung

Vor-AI

- viele Entwickler arbeiteten tiefer in einer Schicht des Stacks
- Expertise war oft an ein bestimmtes Framework oder Systemsegment gebunden
- Übergaben zwischen Frontend, Backend, Datenbank und Betrieb waren alltäglich

Mit AI-Unterstützung

- ein einzelner Entwickler kann heute deutlich leichter über mehrere Schichten hinweg liefern
- Anthropic beschreibt diese Entwicklung explizit als "becoming more full-stack"
- damit wächst auch die Verantwortung für das Gesamtsystem statt nur für ein kleines Teilgebiet

Speaker notes

Traditionell gab es "Frontend", "Backend" und "Datenbank" als isolierte Silos, da jedes Toolset sehr tiefes Spezialwissen erforderte. Mit KI als Übersetzer sinkt die Hürde zwischen diesen Stacks. Daraus entsteht der "True Fullstack Engineer", der Features von der Datenbank bis zur UI Ende-zu-Ende verantwortet.

Warum konzeptionelles Verständnis wichtiger wird

Wichtiger als früher

- Systemgrenzen und Schnittstellen verstehen
- Anforderungen in überprüfbare Teilprobleme zerlegen
- Architektur- und Trade-off-Entscheidungen treffen
- Korrektheits- und Qualitätskriterien explizit machen

Unwichtiger als früher

- jedes Detail eines Nischentools manuell auswendig beherrschen
- Boilerplate und Standardintegration selbst schreiben
- Implementierungswissen mit Engineering-Verantwortung verwechseln

Speaker notes

Die Abstraktionsebene steigt: Statt die genaue Syntax einer Webpack-Konfiguration auswendig zu kennen, müssen Entwickler verstehen, *warum* Bundling, Tree-Shaking und Caching notwendig sind. Das konzeptionelle Wissen ist der Maßstab, gegen den der KI-generierte Code geprüft wird.

Die Aufsicht wird zur Kernaufgabe

Warum Supervision nötig bleibt

- AI kann schnell viel Code erzeugen, aber nicht zuverlässig den fachlichen Kontext ersetzen
- reale Entwicklung enthält Mehrdeutigkeit, implizites Domänenwissen und versteckte Randbedingungen
- Fehler in Architektur, Sicherheit oder Datenflüssen sind teurer als Fehler in Syntax

Befund aus Studien

- Anthropic berichtet, dass die meisten Mitarbeitenden nur `0-20%` ihrer Arbeit vollständig delegieren würden.
- Die ASE-2025-Studie zu Developer-Agent-Collaboration fand mehr Erfolg bei inkrementeller Zusammenarbeit als bei One-shot-Delegation.
- Aktive Iteration, Debugging und Testing mit dem Agenten blieben zentrale Erfolgsfaktoren.

Speaker notes

Künstliche Intelligenz leidet unter Kontextgrenzen und fehlendem Weltwissen über die spezifische Domäne eines Unternehmens. Daher ist One-shot-Generierung (ein Prompt -> fertiges System) in Enterprise-Systemen meist eine Illusion. Die menschliche Rolle ist das "Steering": Der Entwickler korrigiert den Kurs des Agenten anhand inkrementeller Tests.

Neue Schwerpunktkompetenzen

Wichtiger als früher

Kompetenz	Warum
Konzeptverständnis	AI beschleunigt Implementierung, nicht die Systemidee
Problemzerlegung	Gute Delegation setzt gute Problemformulierung voraus
Review & Supervision	AI-Ausgaben müssen verifiziert und eingeordnet werden
Kommunikation	Anforderungen zwischen Menschen, Agenten und Systemen

Unwichtiger als früher

- jedes Detail eines Nischentools auswendig kennen
- Boilerplate selbst schreiben
- Implementierungswissen mit Engineering-Verantwortung verwechseln

Je mehr Coding delegiert wird, desto wichtiger werden **Architektururteil**, **Qualitätsmaßstäbe** und **verantwortliche Aufsicht**.

Speaker notes

Studien von Microsoft und Anthropic zeigen, dass KI vor allem repetitive Implementierungsaufgaben (Boilerplate, Standard-UI) übernimmt. Die Rolle des Engineers wandelt sich zum "Architect & Reviewer". Das kritische Denken über Systemgrenzen, Sicherheit und den fachlichen Kontext bleibt die menschliche Kernaufgabe, die durch KI beschleunigt, aber nicht ersetzt wird.

Vorsicht: Das ist keine vollständige Ersetzung

- Die Studien belegen Produktivitäts- und Delegationsgewinne, aber nicht das Ende menschlicher Softwareentwicklung.
- Die stärksten Aussagen zu "full-stack" und Skill-Verschiebung stammen teilweise von frühen AI-Adoptern; sie sind richtungsweisend, aber nicht automatisch repräsentativ für alle Teams.
- Eine belastbare Schlussfolgerung ist trotzdem: Je mehr Coding delegiert wird, desto wichtiger werden Architektururteil, Qualitätsmaßstäbe und verantwortliche Aufsicht.

> Im Zeitalter von AI verschiebt sich Web Engineering von primär manueller Implementierung hin zu Systemdenken, Delegation, Bewertung und Verantwortung. Gerade deshalb wird Web Engineering wichtiger, nicht weniger.

Speaker notes

Die Befähigung zur Delegation setzt ein tieferes Verständnis des Gesamtsystems voraus. Wer Code generieren lässt, muss in der Lage sein, dessen Qualität, Sicherheit und architektonische Integrität zu beurteilen. Ohne dieses "Urteilsvermögen" führt KI-Automatisierung lediglich zu schnellerer Akkumulation technischer Schulden.

Referenzen

- Microsoft Research, June 2025: *The Effects of Generative AI on High-Skilled Work*
- Anthropic, April 28, 2025: *Anthropic Economic Index: AI's impact on software development*
- ASE 2025: *Why AI Agents Still Need You*
- Anthropic, December 2, 2025: *How AI is transforming work at Anthropic*

Speaker notes

Die aufgeführten Quellen (Microsoft Research, Anthropic, ASE 2025) belegen den Trend zur "Full-Stack"-Verantwortung und die wachsende Bedeutung von Supervision. Sie dienen als Evidenzbasis für die These, dass Web Engineering im AI-Zeitalter anspruchsvoller und konzeptioneller wird.

Zusammenfassung der Evidenzbasis: Die Studien zeigen übereinstimmend, dass KI die Produktivität bei der Routineprogrammierung steigert (bis zu 26%), aber die menschliche Aufsicht bei komplexen Randbedingungen und architektonischen Entscheidungen unverzichtbar bleibt.

Web-Systeme und Daten

Leitfrage: Warum gehört in dieser Veranstaltung zur Web-Perspektive ausdrücklich auch eine Datenperspektive?

Ein konkretes Beispiel: Der personalisierte Feed

Was der Nutzer sieht: eine personalisierte Startseite mit Empfehlungen - scheinbar einfach.

Vereinfachter Fluss

1. Der Nutzer sendet einen HTTP-Request.
2. Ein Load Balancer oder API Gateway leitet den Request weiter.
3. Der Recommendation Service liest Features aus einer Data Pipeline.
4. Ein ML-Modell erzeugt ein Ranking.
    - Das ML-Modell muss vorab trainiert werden
    - Das Training muss auf neuen Daten regelmäßig wiederholt werden
5. Der Service liefert die Empfehlungen über CDN und Frontend aus.

Web und Daten sind zwei Seiten derselben Medaille - ohne Datenpipeline kein Feed, ohne skalierbare Web-Architektur keine Auslieferung.

Speaker notes

Ein personalisierter Feed illustriert die Kopplung von Web und Daten: Der Web-Request stößt eine Kette von Datenabrufen an. Die Architektur muss sicherstellen, dass die Data Pipeline Features (Nutzerpräferenzen, Trends) in Millisekunden liefert, damit die Web-App reaktionsschnell bleibt. Datenarchitektur ist hier eine Performance-Voraussetzung.

Vom Web zum Data Engineering

Warum Data Engineering dazugehört

- Web-Systeme erzeugen und konsumieren große Datenmengen
- Daten müssen gespeichert, verarbeitet und bereitgestellt werden
- moderne Anwendungen kombinieren APIs, Persistenz und Analyse

> **Konsequenz:**
>
> - viele Produktfunktionen hängen heute direkt von Datenqualität und Pipeline-Design ab
> - damit wird Data Engineering zum integralen Teil moderner Web-Systeme

Speaker notes

Die Kopplung von Produkt und Infrastruktur bedeutet, dass Datenarchitektur kein nachgelagerter Schritt ("später mal") oder eine zusätzliche Aufgabe ist. Sie ist integraler Bestandteil des Systementwurfs, da die Verfügbarkeit und Qualität von Daten direkt die Funktionalität und den Wert der Web-Anwendung bestimmen.

Leitfragen für die nächsten Kapitel

Kapitel	Frage
2	Wie ist die Anwendung durch Architektur und Schichten strukturiert?
3	Welche technischen Grundlagen liefern HTTP, HTML und CSS?
4	Wie werden Web-Systeme serverseitig umgesetzt?
5	Wie werden Web-Prinzipien auf APIs übertragen?
6	Wie werden Datenflüsse und Datenprodukte systematisch betrieben?

Speaker notes

Diese Agenda spannt den Bogen von abstrakter Netzwerkarchitektur über konkrete Protokolle (HTTP) und Implementierung (Java) bis hin zur nachgelagerten Datenverarbeitung, und spiegelt somit den gesamten Lebenszyklus einer Eingabe wider.

Takeaway

Die Motivation der Veranstaltung ist doppelt: Web-Systeme sind technisch und organisatorisch komplex, und ihr Wert entsteht immer häufiger aus dem Zusammenspiel mit Daten. Dieser Kurs behandelt beides als zusammenhängende Disziplin.

Speaker notes

Die zentrale Erkenntnis ist die Synergie: Gutes Web Engineering ermöglicht erst die Nutzung großer Datenmengen, und Data Engineering macht Web-Systeme intelligent. Wer beide Disziplinen beherrscht, kann Systeme bauen, die nicht nur technisch funktionieren, sondern durch Daten einen echten Mehrwert für Nutzer und Unternehmen generieren.